In [10]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import confusion_matrix, classification_report

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, LSTM, RepeatVector, TimeDistributed, Dense

# Load dataset
df = pd.read_csv("NASA TurboFan Engine.csv")

# Sort data by engine and cycle
df = df.sort_values(["unit", "cycle"]).reset_index(drop=True)

# Select sensor features
features = [col for col in df.columns if col.startswith("sensor")]

print(df.head())
print("Sensor features:", features)
```

```
   unit  cycle  setting1  setting2  setting3  sensor1  sensor2  sensor3  \
0     1      1   -0.0007   -0.0004     100.0   518.67   641.82  1589.70
1     1      2    0.0019   -0.0003     100.0   518.67   642.15  1591.82
2     1      3   -0.0043    0.0003     100.0   518.67   642.35  1587.99
3     1      4    0.0007    0.0000     100.0   518.67   642.35  1582.79
4     1      5   -0.0019   -0.0002     100.0   518.67   642.37  1582.85

   sensor4  sensor5  ...  sensor14  sensor15  sensor16  sensor17  sensor18  \
0  1400.60    14.62  ...    8138.62    8.4195      0.03       392      2388
1  1403.14    14.62  ...    8131.49    8.4318      0.03       392      2388
2  1404.20    14.62  ...    8133.23    8.4178      0.03       390      2388
3  1401.87    14.62  ...    8133.83    8.3682      0.03       392      2388
4  1406.22    14.62  ...    8133.80    8.4294      0.03       393      2388

   sensor19  sensor20  sensor21  RUL  Fault_Label
0     100.0     39.06   23.4190  191            0
1     100.0     39.00   23.4236  190            0
2     100.0     38.95   23.3442  189            0
3     100.0     38.88   23.3739  188            0
4     100.0     38.90   23.4044  187            0

[5 rows x 28 columns]
Sensor features: ['sensor1', 'sensor2', 'sensor3', 'sensor4', 'sensor5', 'sensor6', 'sensor7', 'sensor8', 'sensor9', 'sensor1
0', 'sensor11', 'sensor12', 'sensor13', 'sensor14', 'sensor15', 'sensor16', 'sensor17', 'sensor18', 'sensor19', 'sensor20', 'se
nsor21']
```

In [11]:

```python
window_size = 24

X_seq = []
y_seq = []

# Create windows separately for each engine
for unit in df["unit"].unique():
    unit_data = df[df["unit"] == unit]

    unit_X = unit_data[features].values
    unit_y = unit_data["Fault_Label"].values

    for i in range(len(unit_X) - window_size + 1):
        X_seq.append(unit_X[i:i + window_size])
```

```
        # A window is faulty if any reading in it is faulty
        y_seq.append(int(np.any(unit_y[i:i + window_size])))

X_seq = np.array(X_seq)
y_seq = np.array(y_seq)

# Split windows and labels together
split = int(0.8 * len(X_seq))

x_train = X_seq[:split]
x_test = X_seq[split:]

y_train = y_seq[:split]
y_test = y_seq[split:]

print("Training windows:", len(x_train))
print("Training labels:", len(y_train))
print("Testing windows:", len(x_test))
print("Testing labels:", len(y_test))
```

```
Training windows: 14664
Training labels: 14664
Testing windows: 3667
Testing labels: 3667
```

In [12]:

```
# Fit scaler using training data only
scaler = MinMaxScaler()
scaler.fit(x_train.reshape(-1, x_train.shape[2]))

# Scale training data
x_train = scaler.transform(
    x_train.reshape(-1, x_train.shape[2])
).reshape(x_train.shape)

# Scale testing data
x_test = scaler.transform(
    x_test.reshape(-1, x_test.shape[2])
).reshape(x_test.shape)

# Keep only healthy windows for training
x_train = x_train[y_train == 0]
```

```
print("Healthy training windows:", len(x_train))
```

```
Healthy training windows: 12153
```

In [13]:
```
model = Sequential([
    Input(shape=(window_size, x_train.shape[2])),
    LSTM(32, activation="relu"),
    RepeatVector(window_size),
    LSTM(32, activation="relu", return_sequences=True),
    TimeDistributed(Dense(x_train.shape[2]))
])

model.compile(optimizer="adam", loss="mse")

history = model.fit(
    x_train,
    x_train,
    epochs=10,
    batch_size=32,
    validation_split=0.1
)
```

```
Epoch 1/10
342/342 ━━━━━━━━━━━━━━━━━━━━ 9s 16ms/step - loss: 0.0152 - val_loss: 0.0049
Epoch 2/10
342/342 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - loss: 0.0043 - val_loss: 0.0047
Epoch 3/10
342/342 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - loss: 0.0042 - val_loss: 0.0047
Epoch 4/10
342/342 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - loss: 0.0042 - val_loss: 0.0046
Epoch 5/10
342/342 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - loss: 0.0042 - val_loss: 0.0046
Epoch 6/10
342/342 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - loss: 0.0041 - val_loss: 0.0046
Epoch 7/10
342/342 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - loss: 0.0041 - val_loss: 0.0046
Epoch 8/10
342/342 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - loss: 0.0041 - val_loss: 0.0045
Epoch 9/10
342/342 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - loss: 0.0041 - val_loss: 0.0045
Epoch 10/10
342/342 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - loss: 0.0041 - val_loss: 0.0045
```

In [14]:
```python
# Reconstruction error on healthy training data
train_pred = model.predict(x_train, verbose=0)

train_error = np.mean(
    np.square(x_train - train_pred),
    axis=(1, 2)
)

# Set fault threshold
threshold = train_error.mean() + 3 * train_error.std()

print("Fault Threshold:", threshold)
```

```
Fault Threshold: 0.009877749491711477
```

In [15]:
```python
# Reconstruction error on test data
test_pred = model.predict(x_test, verbose=0)

test_error = np.mean(
    np.square(x_test - test_pred),
```

```
    axis=(1, 2)
)

# Predict faults
y_pred = (test_error > threshold).astype(int)

print("Detected Faults:", np.sum(y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))
```

```
Detected Faults: 28

Confusion Matrix:
[[3050   28]
 [ 589    0]]

Classification Report:
              precision    recall  f1-score   support

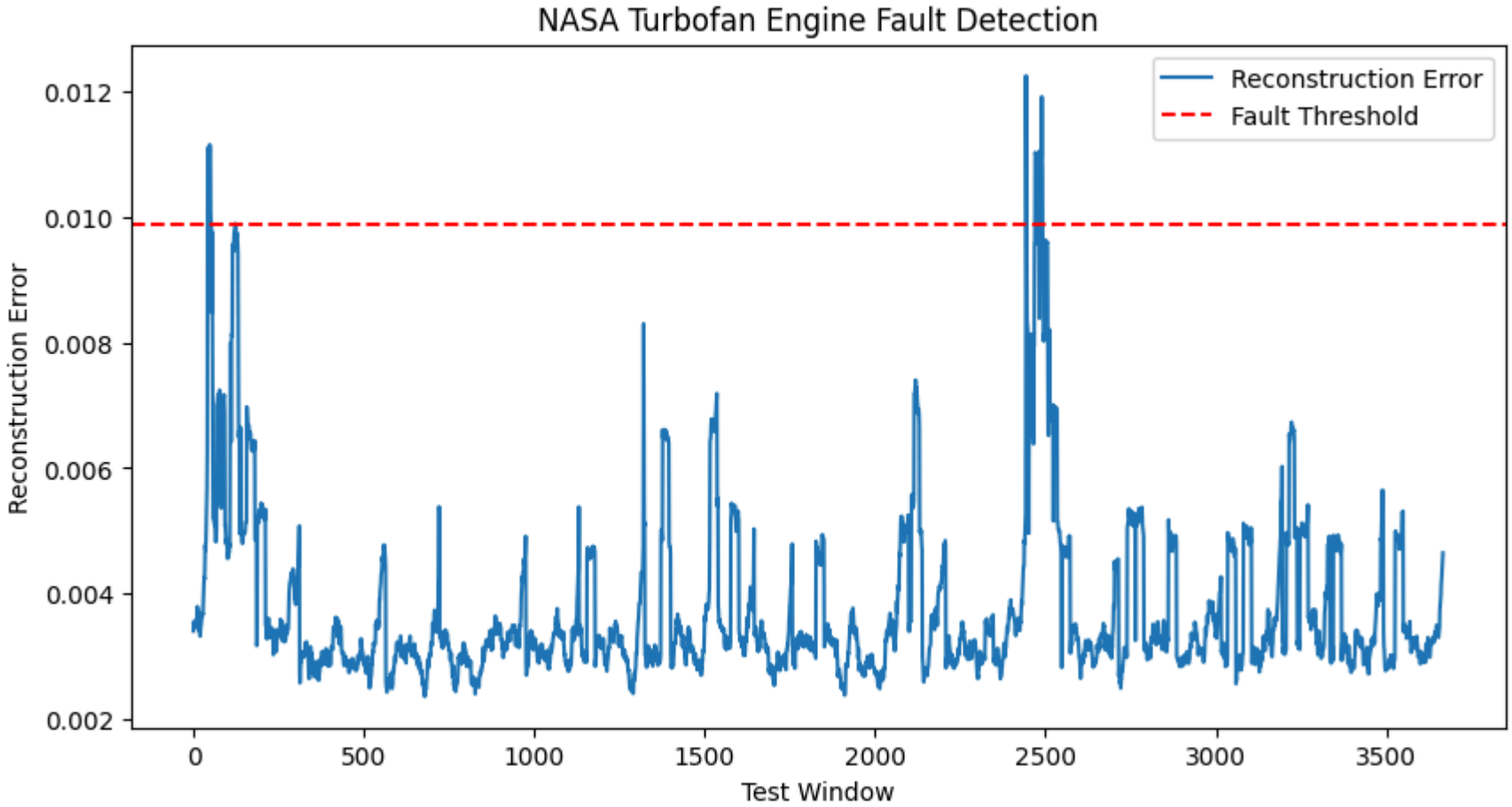
           0       0.84      0.99      0.91      3078
           1       0.00      0.00      0.00       589

    accuracy                           0.83      3667
   macro avg       0.42      0.50      0.45      3667
weighted avg       0.70      0.83      0.76      3667
```

In [16]:
```
plt.figure(figsize=(10, 5))

plt.plot(test_error, label="Reconstruction Error")
plt.axhline(threshold, color="red", linestyle="--", label="Fault Threshold")

plt.xlabel("Test Window")
plt.ylabel("Reconstruction Error")
plt.title("NASA Turbofan Engine Fault Detection")
plt.legend()
plt.show()
```


NASA Turbofan Engine Fault Detection
Reconstruction Error
Fault Threshold
Reconstruction Error
0.012
0.010
0.008
0.006
0.004
0.002
0
500
1000
1500
2000
2500
3000
3500
Test Window

In [ ]: